

Project #2

TurtleBot3 AutoRace: Lane-Following Speed Challenge

Capstone Design [202601-ICE/CSE4020]

Professor: Dr. Oybek Eraliev

EA: Jasurbek Mamurov



1. Overview

In this project, you will program a TurtleBot3 to autonomously drive around a physical race track as **fast as possible** while staying within the lane markings. This is based on the **ROBOTIS AutoRace** competition format, which uses camera-based perception to enable autonomous driving. Unlike Project #1 — which was about reactive navigation and decision-making with LiDAR — this project focuses on **computer vision and lane following**.

There are no traffic lights, signs, obstacles, or parking zones in this challenge. The single mission: **complete a lap of the physical track in the shortest possible time, with the fewest line crossings**.

2. Background and Tools

You are not expected to master everything from scratch — the official ROBOTIS package provides a working baseline that you will study, tune, and improve.

2.1 The AutoRace 2020 Package

ROBOTIS provides an official open-source package called `turtlebot3_autorace_2020` that contains everything needed for camera-based lane following: camera calibration, lane detection (using OpenCV color filtering and perspective transformation), and a control loop that converts the detected lane into steering commands. **You will use this package as your baseline.** Your task is to understand it, adapt it to your specific physical track, and tune it for racing performance.

Reference: https://github.com/ROBOTIS-GIT/turtlebot3_autorace_2020

2.2 How Lane Following Works

The lane-following pipeline involves several stages working together. First, **camera calibration** corrects lens distortion and computes a perspective transformation that converts the camera's angled view into a top-down "bird's-eye view" of the road ahead. Next, **color filtering** uses HSV thresholds to extract the yellow and white lane markings from the image. Then, **lane detection** analyzes the filtered image to identify each lane line and computes the **lane center** (the

midpoint between the two lines). Finally, the **control loop** compares the lane center to the robot's desired position (the image center) and computes a steering correction — typically using a proportional controller.

All of these stages have **tunable parameters** that affect performance: HSV thresholds determine which colors are detected as lane markings; perspective transformation parameters determine the bird's-eye view geometry; control gains determine how aggressively the robot steers. Tuning these for your specific physical track is a major part of this project.

2.3 Role of Gazebo and RViz

Gazebo is essential for development. The AutoRace package includes a Gazebo world with a built-in lane-following track that mirrors the physical track's structure. You should do most of your initial development and algorithm work in Gazebo before moving to the physical robot — it is much faster to iterate in simulation, and you can crash, reset, and try again endlessly without consequences.

RViz is essential for debugging on the physical robot. While the TurtleBot3 drives the track, you can run RViz on your laptop to visualize the camera feed, the color-filtered images, the bird's-eye projection, and the detected lane lines in real time. This is the difference between a robot that **inexplicably fails** and one whose failure modes you can actually understand and fix.

2.4 Useful References

- **AutoRace e-Manual** — https://emanual.robotis.com/docs/en/platform/turtlebot3/autonomous_driving/
- **AutoRace 2020 GitHub** — https://github.com/ROBOTIS-GIT/turtlebot3_autorace_2020
- **OpenCV color spaces (HSV)** — https://docs.opencv.org/4.x/df/d9d/tutorial_py_colorspaces.html
- **ROS image_transport** — http://wiki.ros.org/image_transport
- **TurtleBot3 e-Manual (Bringup)** — <https://emanual.robotis.com/docs/en/platform/turtlebot3/bringup/>

3. Foundational Setup

This is a brief checklist of essential setup steps. Detailed instructions are available in the AutoRace e-Manual referenced above — you are expected to follow those for the full procedure.

1. **Install the AutoRace 2020 package** into your existing catkin workspace:

```
cd ~/catkin_ws/src
git clone -b noetic https://github.com/ROBOTIS-GIT/\
  turtlebot3_autorace_2020.git
cd ~/catkin_ws && catkin_make
```

2. **Install required dependencies** on the Remote PC:

```
sudo apt install ros-noetic-image-transport \
  ros-noetic-cv-bridge ros-noetic-vision-opencv \
  python3-opencv libopencv-dev \
  ros-noetic-image-proc
```

3. Set the TurtleBot3 model to `burger_cam` (the camera-equipped version):

```
echo 'export TURTLEBOT3_MODEL=burger' >> ~/.bashrc
source ~/.bashrc
```

4. **Test the simulation.** Launch the AutoRace Gazebo world and verify that the simulated track loads correctly with a TurtleBot3 in it.
5. **Set up the physical robot.** Follow the standard TurtleBot3 bringup procedure to connect your robot to the Remote PC over Wi-Fi. Verify that the camera feed is published correctly by viewing the `/camera/image` topic in `rqt` or `RViz`.
6. **Run the baseline lane-following code** in simulation first. This gives you a reference behavior to compare your improvements against.

4. The Main Task

Your team's goal is to produce a TurtleBot3 capable of completing a full lap of the physical track autonomously, as quickly as possible, while crossing the lane lines as few times as possible. The work breaks down into the following components.

4.1 Understand the Baseline

Read through the AutoRace 2020 source code and understand each stage of the lane-following pipeline. You should be able to explain in your report and in the Q&A what each parameter does and why you chose the values you did.

4.2 Adapt the Pipeline to the Physical Track

The baseline parameters are tuned for a specific lighting and color setup that will not match your physical track exactly. You will need to:

- **Recalibrate the camera** (intrinsic and extrinsic) on the physical track to get an accurate bird's-eye view.
- **Tune HSV color thresholds** for your track's actual lane colors under your room's actual lighting. The values that work in Gazebo will likely not work directly on the physical track.
- **Adjust the perspective transformation** parameters so that the bird's-eye view shows a useful section of the track ahead.
- **Tune the control gains** so the robot follows lanes smoothly without oscillating, while still being able to steer through curves.

4.3 Optimize for Speed

Once your robot can reliably follow the track, the optimization phase begins. This is where the real engineering happens. Consider:

- **Linear velocity** — The default speed in the baseline is conservative. Increase it gradually and observe how the robot handles curves at higher speeds.
- **Adaptive speed** — You can implement logic that slows down on sharp turns and accelerates on straight sections, similar to how a human driver behaves.
- **Look-ahead distance** — Adjusting how far ahead the robot “sees” affects how it anticipates curves. Looking too close causes late steering; looking too far loses precision.
- **Control loop frequency** — Faster control loops give smoother trajectories but use more CPU. Find the right balance.

4.4 Test, Iterate, Refine

Test extensively in Gazebo first — your simulation lap time is a useful baseline. Then move to the physical track and expect a significant gap between simulation and reality. Document each iteration: what you changed, what improved, what got worse. This documentation will be the backbone of your written report.

Tips and Strategy

- Develop in Gazebo first. Most algorithm bugs are easier to find in simulation than on a physical robot that drives off the track.
- Use RViz constantly during physical testing. Watching the camera feed and lane detection in real time tells you exactly why the robot is misbehaving.
- Test under different lighting conditions. Morning vs evening lighting in your testing room can change which HSV values work. Tune for the conditions expected on demo day.
- Increase speed in small steps. Going from 0.2 m/s to 0.4 m/s is a big change — the control loop may need re-tuning at higher speeds.
- Save your parameter sets. Maintain a YAML or text file with the parameter combinations you have tried and their results.
- Divide the work. With 6–8 members, split into subgroups: simulation tuning, physical testing, perception (HSV/calibration), control (steering/speed), and so on. Keep in mind that only preparing for a presentation or only preparing a report is not sufficient work done in a team.
- Charge your TurtleBot3 batteries fully before each test session and especially before demo day. Low battery affects motor performance and can cause unpredictable behavior.

5. Live Demonstration and Q&A

On demonstration day, each team will perform a **single attempt** on the physical track.

5.1 Demo Logistics

- **All team members must be present** during the demonstration.
- Only **one designated team member** may interact with the robot during the run.
- The run starts when the robot is placed at the announced **start position** on the track.
- The exact start position and direction of travel will be announced on demo day.
- **Recording is mandatory.** Each team must record two synchronized videos and upload both to **YouTube** (links included in the report):
 - **Computer screen recording** showing the camera feed, lane detection visualization (RViz or rqt_image_view), and terminal output during the run.
 - **Physical robot recording** showing the TurtleBot3 driving the track from an external camera angle. This must cover the entire run from start to finish.
- The two videos must clearly correspond to the same run. Mismatched or partial recordings will cause trouble.

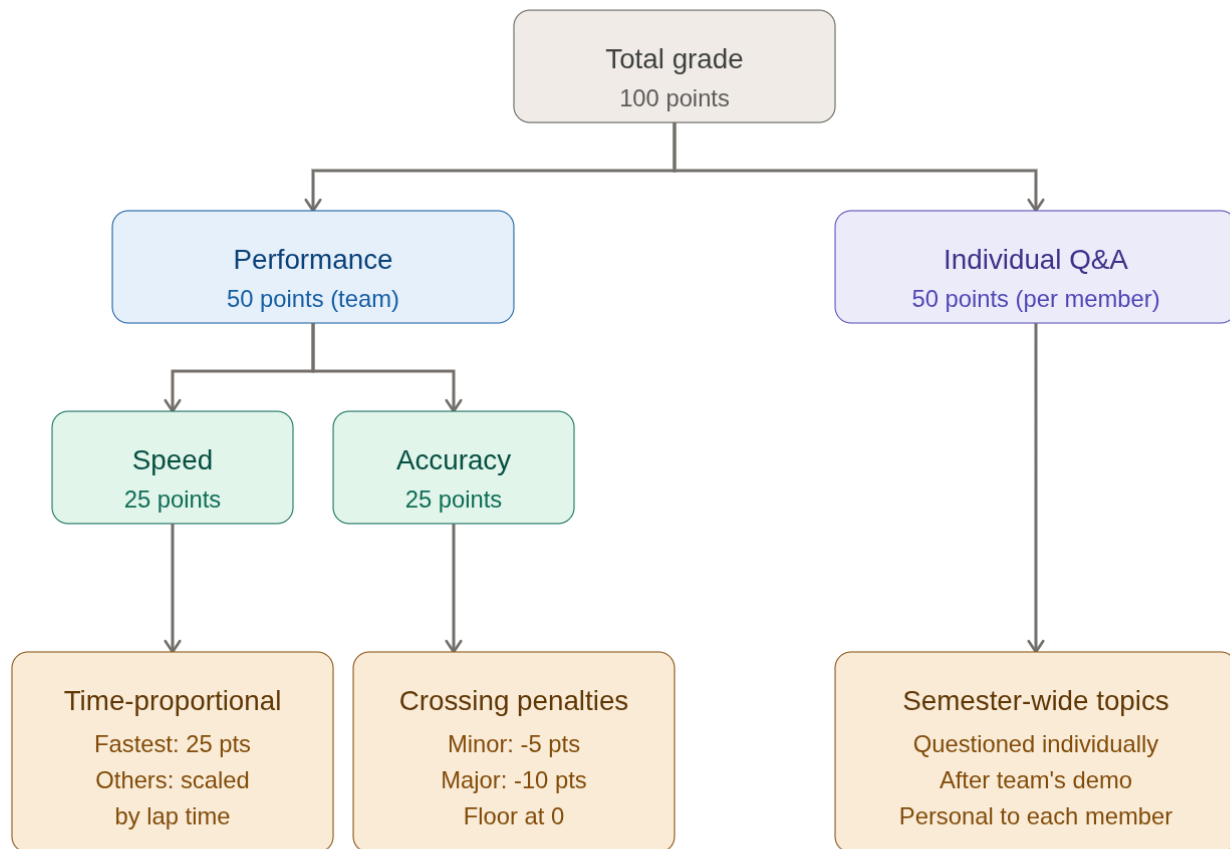
5.2 Q&A Session

Immediately after each team's demonstration, there will be a **Q&A session** where the course staff and Education Assistant ask the team questions about the topics covered during the semester.

Note: Try to avoid surface-level answers!

6. Grading Rubric

Your submission will be evaluated according to the criteria below **and will be mostly relative**. The total score is 100 points.



7. Submission Instructions

Your team must submit the following items through the specified platforms.

1. **Source code and configuration files** — Submit your complete ROS package, including any modified launch files, calibration YAMLS, parameter files, and your Python or C++ source code, to the **GitHub repository of IUT** designated for this course. The repository must include a [README.md](#) explaining how to build and run your solution.
2. **Written report (PDF, 3–6 pages)** — Submit to **e-Class**. The report must include:
 - **Approach and Architecture** — description of your lane-following pipeline, parameter choices, and any modifications to the baseline.
 - **Parameter Choices** — the values you settled on (HSV thresholds, control gains, speed) and why.
 - **Challenges and Solutions** — specific problems encountered and how you addressed them.

- **Results** – lap time, line crossings observed, comparison between simulation and physical performance.
- **Team Contributions** – each member's role and specific contributions.
- **YouTube video links** – both the computer screen recording and the physical robot recording, clearly labeled.

Late submissions are not allowed!

8. Important Notes

1. This is a **team assignment**. All team members are expected to contribute. The report must clearly detail individual contributions.
2. **Beware of the line-crossing penalties!** Each lane-line crossing during the run is penalized. [Deduction: **-5 points** per minor crossing, **-10 points** per major crossing – *going farther away from the line.*]
3. You are encouraged to use the official **AutoRace 2020 baseline code** as your starting point. Building from scratch is not required and is not recommended due to limited time availability. However, you must understand the code and document your modifications.
4. Treat the hardware with care. **No components must be removed from a TurtleBot!** Damaged components must be reported immediately to the Education Assistant. Negligent damage may result in trouble.
5. Be respectful of other teams sharing the track space. Coordinate testing times if needed and keep the area clean.
6. Sharing code between teams is not permitted. Each team's solution must be developed independently. Reusing the official ROBOTIS baseline is allowed and encouraged – your modifications, parameter tuning, and any custom logic must be original.
7. Charge batteries fully before the demo. A discharged TurtleBot3 will perform poorly regardless of code quality.
8. Plan for the unexpected. Demo day conditions (lighting, audience movement, robot temperature) may differ from your testing. Build robustness into your solution.

Good luck, and may the fastest team win!
For any questions, reach out to the Education Assistant of this course.